

Décisions d'architecture

Décisions techniques (ADR) retenues pour le projet et justifications. Le détail de la recherche comparative qui a nourri ces décisions se trouve dans [benchmark-techno.md](#) (même dossier).

| VUE D'ENSEMBLE

Principes

Plusieurs principes ont été suivis dans les choix des technologies. Côté fonctionnel, l'analyse a révélé des usages multiples dans l'exploitation des données, associant la plupart du temps plusieurs outils, en particulier QGIS et Excel. Les données peuvent provenir de la base de données de l'Audiar (bdsig), de ses ressources partagées (Z: et K:) ou directement de la source (web, producteur).

Comme indiqué dans l'**analyse technique**, plusieurs critères ont été formulés par le pôle Données pour sélectionner la solution d'orchestration :

- open source et gratuits ;
- maintenance minimale ;
- pouvant regrouper le maximum de fonctionnalités au vu des cas d'usages ;
- IA agnostique (n'importe quel fournisseur API peut être utilisé) ;
- gestion des utilisateurs possibles ;
- connexion possibles à des MCP ;
- fiabilité (documentation, pérennité de la technologie)

Plusieurs autres éléments – opportunités ou contraintes –, ont aussi influencé les choix de technologie et les scénarios de développement :

- possibilité de déployer un outil sur un serveur local (Merlin) ;
- démarche de montée en compétences en python pour le traitement de données ;
- accès à l'API de RAGaRenn et possibilité de se tourner vers OVH Cloud.
- catalogage de bdsig non terminé ;
- chantier d'intégrations de données en cours (pôles environnement et habitat-démographie notamment).

Récapitulatif des décisions

Composant	Outil retenu	Statut (30/07/2026)	Détail
Solution d'orchestration	Open WebUI	Deux instances créées : prototype local (poste Charlotte) et prototype Merlin	Section « Solution d'orchestration » ci-dessous
Connexion des serveurs MCP	mcpo (proxy MCP → OpenAPI)	Choix arrêté	Section « Connexion des serveurs MCP à Open WebUI » ci-dessous
Utilisation QGIS	MCP local nkarasiak/qgis-mcp	Configuré sur Merlin, en test	servers/mcp-qgis/README.md
Accès base de données	outil python	Monté sur Merlin, à optimiser	
Utilisation Excel	MCP haris-musa/excel-mcp-server	Pas encore monté (dossier servers/mcp-excel/ à créer)	Section « Serveurs MCP retenus » ci-dessous
Recherche web (fonction native Open WebUI)	Brave, SearXNG et Tavily benchmarkés	À l'étude	benchmark-techno.md , section « Moteurs de recherche web candidats »

SOLUTION D'ORCHESTRATION

Solutions prête à l'emploi versus frameworks

Deux approches possibles pour construire l'outil : une solution prête à l'emploi (peu ou pas de code, cf. [benchmark-techno.md](#) section « Solutions prêtes à l'emploi »), ou un framework à assembler soi-même (LangGraph, CrewAI... cf. même fichier, section « Frameworks »), à l'image de l'application de transcription développée par le pôle Données.

Décision : solution prête à l'emploi. Le critère « maintenance minimale » a pesé plus lourd que la flexibilité d'un framework, qui fait porter au pôle Données la maintenance dans la durée de tout ce qu'une solution mature fournit déjà : gestion des utilisateurs/droits, mémoire/persistance, traçabilité, interface. Construire cela soi-même avec LangGraph (l'un des framework les plus pertinents du comparatif) reviendrait à réimplémenter une partie d'Open WebUI.

Solution d'orchestration : Open WebUI Audiar

Décision (20/07/2026) : pour l'usage courant (catalogue, appels d'outils MCP dont QGIS en mode hybride — cf. [servers/mcp-qgis/README.md](#)), le framework retenu est **Open WebUI** (« Open WebUI Audiar » une fois l'instance créée et personnalisée). Aucune instance Open WebUI n'existe encore chez Audiar à ce stade — elle est à créer, en commençant par une installation locale sur le poste de Charlotte (prototype, voir [docs/guides.md](#)), avant d'envisager un hébergement partagé pour plusieurs chargés d'études. Comparé aux dix autres candidats évalués (voir [benchmark-techno.md](#)), justification par critère :

- **Effort de développement** : plus faible que les autres candidats du benchmark, mais pas nul — l'instance est à créer (voir [docs/guides.md](#), pas-à-pas d'installation locale via Docker). Reste ensuite à connecter les serveurs MCP métier, via le proxy [mcpo](#) (MCP → OpenAPI) — voir [servers/mcp-qgis/README.md](#) pour le détail et la justification de ce choix. Contrairement à un développement sur-mesure (cf. [benchmark-techno.md](#), section briques d'orchestration), il n'y a rien à coder : seulement à déployer et configurer des outils existants.
- **Documentation** : documentation officielle complète et à jour ([docs.openwebui.com](#)), section dédiée à l'extensibilité MCP ([docs.openwebui.com/features/extensibility/mcp/](#)), large communauté donc bon niveau de tutoriels/retours d'expérience tiers.
- **Facilité d'utilisation** : interface de chat déjà connue en interne par les chargés d'études (outil existant, pas un nouvel outil à apprendre) ; prise en main immédiate pour l'usage courant (chat/RAG). La configuration MCP/agentique reste du ressort de l'administrateur, transparente pour l'utilisateur final une fois en place.
- **Outils paramétrables** : compatibilité MCP native depuis la v0.6.31, ou via le proxy [mcpo](#) (MCP → OpenAPI) sur les versions antérieures ; pipelines/« fonctions » Python + large écosystème de plugins communautaires pour étendre au-delà du MCP.
- **Gestion mémoire** : historique des conversations persistant côté serveur + RAG documentaire intégré — pertinent pour le cas d'usage 1 (connaissance du catalogue Audiar).
- **Gouvernance multi-utilisateurs** : RBAC natif, groupes, SSO possible — un des points les plus mûrs du benchmark, nécessaire pour un déploiement à plusieurs chargés d'études (contrairement aux outils mono-utilisateur comme Goose ou VS Code + Cline).
- **Souveraineté** : self-hosted par construction, compatible Ollama et tout endpoint API OpenAI — cohérent avec la contrainte de sobriété/souveraineté des données (RAGaRenn/OVH Cloud).

Point de vigilance conservé : la licence « Open WebUI License » n'est plus certifiée OSI depuis la v0.6.6 (BSD 3-Clause + clause de marque) — à surveiller si l'agence a une politique stricte sur les licences ; des discussions de fork ont déjà eu lieu dans la communauté à ce sujet, sans qu'un fork dominant et pérenne ne se soit encore imposé à ce jour.

Limite assumée : le « mode projet » d'Open WebUI reste plus faible que des candidats comme Goose, Eigent ou Open Cowork (pas d'exécution autonome multi-étapes native, dépend du proxy [mcpo](#) pour l'agentique). Ce choix couvre l'usage courant (chat, catalogue, appels d'outils MCP dont QGIS en mode hybride) ; à réévaluer si un besoin d'agent autonome de type Cowork sur un dossier de travail complet devient prioritaire.

Extensibilité Open WebUI (pipelines/« fonctions » Python) : Open WebUI permet d'étendre son comportement par du code Python custom (pipelines/functions), au-delà du MCP, ou en alternative.

| SERVEURS MCP RETENUS

Comparaison des implémentations candidates pour chaque serveur : voir [benchmark-techno.md](#), section « Serveurs MCP candidats ».

- **mcp-qgis** : [nkarasiak/qgis-mcp](#) — retenu pour ses capacités étendues (100+ outils vs ~15 pour l'implémentation d'origine jjsantos01/qgis_mcp) et sa compatibilité avec le mode hybride (agit sur le projet QGIS ouvert à l'écran). Détail du montage technique complet : [servers/mcp-qgis/README.md](#) . **Statut (20/07/2026) : monté et validé**, y compris pour un compte non-admin.
- **mcp-postgres** : [crystaldba/postgres-mcp](#) (« Postgres MCP Pro ») — retenu plutôt que l'ancienne implémentation de référence Anthropic (archivée, faille d'injection SQL documentée). Détail : [servers/mcp-postgres/README.md](#) . **Statut (20/07/2026) : prochain serveur à monter** — répond au cas d'usage 2 de l'analyse fonctionnelle (« extraction de données depuis une table en base », outils envisagés « Open WebUI + MCP BD »).
- **mcp-filesystem** : implémentation de référence officielle [modelcontextprotocol/servers](#) — la plus pérenne des candidates recensées (maintenue par le MCP steering group/Anthropic). Détail : [servers/mcp-filesystem/README.md](#) . **Statut (20/07/2026) : reporté, pas de priorité dans le scope Open WebUI actuel.** Les cas d'usage 8/9 de [docs/sources/2026_analyse-fonctionnelle_V1.txt](#) (« mode projet », accès à l'ensemble des fichiers) visent VS Code + Cline, Claude Desktop ou un outil type Cowork open source — pas Open WebUI, cohérent avec la « Limite assumée » ci-dessus sur le mode projet d'Open WebUI. [mcp-filesystem](#) n'a donc de sens que si un outil « mode projet » distinct d'Open WebUI est un jour retenu pour ce scope. (Ne concerne que l'accès fichiers générique — voir [mcp-excel](#) ci-dessous pour le cas Excel, qui suit un raisonnement différent.)
- **mcp-excel** : [haris-musa/excel-mcp-server](#) — le plus populaire et actif des deux candidats évalués, ne nécessite pas Excel installé. Pas encore de dossier [servers/mcp-excel/](#) créé à ce stade (serveur pas encore mis en place). **Décision (22/07/2026) : exposé via Open WebUI/ [mcpo](#)**, comme les autres serveurs MCP métier du projet — priorité donnée au regroupement du plus grand nombre de cas d'usage dans un seul outil plutôt qu'à la dispersion entre plusieurs interfaces. Le cas d'usage 3 de [docs/sources/2026_analyse-fonctionnelle_V1.txt](#) recommandait un contrôle pas à pas via une extension applicative ou Claude Desktop plutôt qu'Open WebUI pour ce cas précis (risque : erreurs dans les formules générées, perte de maîtrise si l'utilisateur ne comprend pas le traitement produit) — ce point de vigilance reste valable en usage (vérification des résultats à prévoir) mais ne remet pas en cause le choix de passer par Open WebUI. Voir aussi « Point de vigilance : mode hybride pour mcp-excel » ci-dessous.

POINT DE VIGILANCE : MODE HYBRIDE POUR MCP-EXCEL (VOIR LES MODIFICATIONS DANS LE FICHIER OUVERT PAR L'UTILISATEUR)

Pas encore tranché (identifié le 22/07/2026). Par analogie avec le mode hybride retenu pour [mcp-qgis](#) (agir sur le projet ouvert à l'écran, cf. [servers/mcp-qgis/README.md](#)), un besoin équivalent est identifié pour Excel : pouvoir vérifier rapidement les modifications faites par l'IA sur un fichier que le chargé d'études a déjà ouvert, sans devoir le télécharger/rouvrir/comparer manuellement avec l'original — à la fois pour le contrôle (cas d'usage 3, risque documenté d'erreurs dans les formules générées) et pour l'ergonomie (gain de temps de vérification).

Cadrage retenu : les fichiers Excel concernés sont des fichiers **locaux**, sur le poste Windows de chaque chargé d'études — pas des fichiers hébergés sur OneDrive/SharePoint. La piste d'une co-édition via Microsoft Graph API (fichier en ligne, co-édition native Excel) est donc écartée d'emblée pour ce prototype, cohérente avec la contrainte de souveraineté déjà notée pour les données sensibles (cas d'usage 3 : « solution souveraine, OVH Cloud/RAGaRenn »).

Deux familles d'approches en présence, pas encore arbitrées — comparatif détaillé à mener dans [benchmark-techno.md](#), section « mcp-excel — mode hybride » :

- **Édition live via automatisations Windows (COM/OLE)** — pilote la session Excel réellement ouverte par l'utilisateur, modifications visibles immédiatement, sur le même principe que le mode hybride QGIS. `negokaz/excel-mcp-server` (deuxième candidat déjà évalué, voir `benchmark-techno.md`) propose une fonctionnalité « **Live editing** » + **capture d'écran, Windows uniquement, via un backend OLE** — piste la plus directement comparable. `sbroenne/mcp-server-excel` (jusqu'ici écarté du comparatif pour une raison à corriger, voir `benchmark-techno.md`) repose sur le même principe via l'API COM complète. Risque à surveiller, déjà rencontré sur QGIS : les outils structurés basés sur un pilotage live peuvent être moins fiables qu'un outil de fichier direct, avec un rebond possible sur de l'exécution de code brut (cf. `servers/mcp-qgis/README.md`, section « Points de vigilance »).
- **Écriture directe du fichier (openpyxl) + restitution facilitant la vérification** — approche des candidats actuellement priorisés (`haris-musa`, `guillehr2`), sans lien avec une session Excel ouverte. Plus robuste a priori (pas de dépendance à l'automatisation d'une application), mais deux limites à vérifier concrètement : (1) un fichier ouvert dans Excel est en général verrouillé en écriture sous Windows — à confirmer si cela bloque l'écriture par l'outil tant que le fichier reste ouvert, ce qui reproduirait la friction qu'on cherche à éviter ; (2) sans mode live, la vérification devrait passer par un mécanisme de substitution (changelog des cellules modifiées, surlignage) plutôt que par une vue directe du fichier ouvert.

CONNEXION DES SERVEURS MCP À OPEN WEBUI : POURQUOI MCPo

Chaque serveur MCP métier (`mcp-qgis`, `mcp-postgres`, futurs `mcp-filesystem` / `mcp-excel`) est exposé à Open WebUI via **mcpo** ([open-webui/mcpo](https://open-webui.com/mcpo), proxy MCP stdio → OpenAPI), ajouté ensuite comme connexion **OpenAPI** — jamais en MCP natif, malgré le support MCP natif d'Open WebUI depuis la v0.6.31. Deux raisons, documentées sur docs.openwebui.com/features/extensibility/mcp et vérifiées en conditions réelles le 20/07/2026 (sur `mcp-qgis`, premier serveur monté) :

1. **Admin-only et centralisé** : un serveur MCP natif ne peut être ajouté que par un administrateur (*Admin Settings* → *External Tools*), jamais par un utilisateur individuel. Sur une instance mutualisée entre plusieurs chargés d'études, une URL `localhost` configurée une fois par l'admin pointerait vers le poste de l'admin, pas vers celui de chaque agent — inutilisable pour un montage où l'outil tourne sur le poste de chaque utilisateur (ex. QGIS en mode hybride, cf. `servers/mcp-qgis/README.md`). Le mécanisme qui résout ce problème pour un déploiement à plusieurs utilisateurs — « **Direct Tool Servers** », qui fait bien partir l'appel depuis le navigateur de chacun pour atteindre son propre `localhost` — n'existe que pour les serveurs **OpenAPI**, pas pour MCP natif. D'où le choix d'exposer chaque serveur MCP métier en OpenAPI via `mcpo`. Paramétrage des droits nécessaire côté Open WebUI pour activer ce mécanisme : voir `docs/guides.md`, section « Paramétrer les droits Open WebUI (instance mutualisée) ».
2. **Protection anti-DNS-rebinding, même en local** : les serveurs MCP construits avec FastMCP (dont `qgis-mcp-server`) activent par défaut une vérification de l'en-tête HTTP `Host` de la requête entrante contre une liste blanche limitée à `localhost` / `127.0.0.1`. Comme Open WebUI tourne dans Docker, il doit appeler `http://host.docker.internal:<port>` pour atteindre le poste hôte — un `Host` qui n'est jamais dans cette liste blanche. Résultat observé (sur `qgis-mcp-server`) : `421 Misdirected Request`. Aucune variable d'environnement documentée par ce projet ne permet d'élargir cette liste blanche. Ce blocage touche donc aussi le cas d'un Open WebUI local (pas seulement l'instance mutualisée), et concerne potentiellement tout futur serveur MCP également construit avec FastMCP.

`mcpo` évite ce deuxième problème par construction : il lance le serveur MCP lui-même en sous-processus (transport `stdio`, pas de couche HTTP entre les deux), et expose sa propre API OpenAPI (FastAPI, sans cette vérification de `Host`).

Global (Admin) vs Direct (personnel) : mécanisme vérifié le 22/07/2026

Le point 1 ci-dessus affirmait déjà que « Direct Tool Servers » fait partir l'appel depuis le navigateur de chacun — désormais vérifié concrètement, en testant la même URL (`http://localhost:8001` , `mcp-qgis`) depuis les deux écrans où une connexion OpenAPI peut être ajoutée :

- **Admin Panel → Settings → Outils → External Tool Servers** (« Global ») : `localhost:8001` échoue (Échec de la connexion). Confirme que l'appel part du **backend/conteneur** Open WebUI — `localhost` y désigne le conteneur lui-même, jamais le poste de l'utilisateur.
- **Réglages personnels → Intégrations → « Gérer les serveurs d'outils »** (« Direct ») : `localhost:8001` réussit (Connexion réussie). Confirme que l'appel part bien du **navigateur** de l'utilisateur — `localhost` y désigne son propre poste, quel que soit l'endroit où tourne le backend Open WebUI (poste local aujourd'hui, Merlin demain).

Règle de choix entre les deux, qui ne dépend pas seulement de l'endroit où tourne l'outil mais de la cible qu'il représente : - **Cible unique, valable pour tout le monde** (ex. un serveur centralisé à identité technique partagée) → chemin **Global**, configuré une fois par l'admin. Fonctionne dès que le backend et l'outil tournent sur le même serveur (ex. futur `mcp-postgres` centralisé sur Merlin). - **Cible propre à chaque utilisateur** (ex. `mcp-qgis` , ou tout serveur nécessitant des identifiants personnels — cf. `docs/guides.md` , section « Gestion des secrets ») → chemin **Direct**, configuré individuellement par chaque utilisateur dans ses réglages personnels, même si l'outil tourne physiquement sur Merlin plutôt que sur son poste.

Piste non vérifiée pour concilier centralisation et identifiants personnels sans faire reconfigurer chaque utilisateur : le bouton « **Accès** » visible sur l'écran *External Tool Servers* (Admin) suggère une restriction d'accès par connexion, comme pour les modèles — permettrait de déclarer N connexions Global (une par utilisateur) tout en limitant chacune à son propriétaire. À tester avant d'en dépendre pour `mcp-postgres` .

Correction associée : l'URL `http://host.docker.internal:8001` documentée jusqu'ici pour la connexion « Direct » de `mcp-qgis` (`servers/mcp-qgis/README.md`) était incorrecte pour ce chemin précis — probablement une confusion avec le chemin Admin/Global, où elle, en revanche, est correcte (le conteneur doit sortir vers l'hôte). Le chemin Direct doit utiliser `http://localhost:8001` , jamais `host.docker.internal` . Point de vigilance non résolu : une fois Open WebUI servi depuis une autre origine que `localhost` (Merlin, avec ou sans HTTPS), le navigateur enverra cette nouvelle origine dans sa requête vers le `mcpo` local de l'utilisateur — `mcpo` / `qgis-mcp-server` doivent l'accepter en CORS, sans quoi la requête est bloquée malgré des identifiants corrects. Non testé au-delà d'un Open WebUI servi en local (origine triviale).

Mode --config de mcpo — **non exploité à ce stade** : en plus de la commande inline utilisée aujourd'hui (un serveur MCP par instance `mcpo` /par port, cf. `servers/mcp-*/start.sh`), `mcpo` propose un mode `--config <fichier.json>` acceptant un fichier au format `mcpServers` (celui de Claude Desktop) déclarant plusieurs serveurs MCP à la fois, chacun exposé sous sa propre route (`/nom-du-serveur`) par une seule instance `mcpo` . Deux options supplémentaires viennent avec ce mode : `disabledTools` (liste de noms d'outils à exclure de ce qui est exposé, par serveur) et `--hot-reload` (recharge automatique si le fichier de config change, sans redémarrer `mcpo`). Non retenu pour l'instant : le besoin actuel (un serveur MCP métier = une instance `mcpo` = un port dédié, cf. `servers/mcp-*/README.md`) reste plus simple à couvrir en mode inline. À reconsidérer si le besoin de grouper plusieurs serveurs MCP derrière un seul `mcpo` , ou de filtrer des outils spécifiques via `disabledTools` , se présente.

LANCEMENT DES SERVEURS MCP : POURQUOI UV / UVX PLUTÔT QU'UN PIP INSTALL CLASSIQUE

Décision : chaque serveur MCP métier (`qgis-mcp-server` , `postgres-mcp`) est lancé via `uvx` — jamais installé « en dur » dans un environnement Python dédié via `pip install` . Commandes exactes : `servers/mcp-qgis/README.md` et `servers/mcp-postgres/README.md` ; pas-à-pas d'installation d' `uv` sur le poste : `docs/guides.md` .

Justification (vérifiée le 20/07/2026 lors du montage de `mcp-qgis`, équivalent de `npj` côté écosystème JavaScript) :

- Pas de `venv` à créer/activer manuellement par un profil non-développeur à chaque session.
- Toujours la version exacte du dépôt GitHub visé (`--from git+https://...`), sans étape d'installation séparée à maintenir dans le temps.
- Cache stocké globalement (`%LOCALAPPDATA%\uv\cache` sous Windows) — rien ne pollue le dossier du projet.
- Téléchargement une seule fois (mise en cache), démarrages suivants rapides ; `--refresh-package` disponible pour forcer une révérification de la dernière version si besoin (non utilisé au quotidien).

POINT DE VIGILANCE : PASSAGE À L'ÉCHELLE (15 UTILISATEURS) — SUPERVISION DES PROCESSUS UV / MCPO

Pas encore tranché (identifié le 21/07/2026). Aujourd'hui, chaque serveur `mcpo / uvx` est lancé manuellement dans un terminal laissé ouvert sur le poste de Charlotte — acceptable pour un prototype à une personne, mais ça ne tiendra pas pour un déploiement à 15 chargés d'études non-développeurs. La réponse diffère selon deux familles de serveurs :

- **Serveurs liés à un poste individuel** (`mcp-qgis` aujourd'hui ; potentiellement de futurs `mcp-excel / mcp-filesystem` s'ils touchent des fichiers ou applications locales) : doivent tourner sur la machine de chaque agent, pas ailleurs. Le geste manuel actuel (ouvrir un terminal, coller la commande, le laisser ouvert) devra être remplacé par un lancement silencieux et automatique — piste à évaluer : tâche planifiée au démarrage/à l'ouverture de session Windows, ou véritable service Windows (ex. NSSM/WinSW encapsulant la commande `uvx`), avec redémarrage automatique en cas de plantage.
- **Serveurs centralisés** (`mcp-postgres` , qui n'a aucune raison de dépendre du poste d'un utilisateur en particulier puisqu'il interroge une base distante) : à déployer une seule fois, sur le même serveur que l'instance Open WebUI mutualisée à venir, géré comme un vrai service (conteneur Docker avec `--restart always` , ou service systemd/Windows) plutôt qu'en commande manuelle dans un terminal lié à un poste précis. **Correction (24/07/2026)** : cette phrase supposait implicitement une identité de service partagée pour `mcp-postgres` . Ce n'est pas le cas actuellement (chaque chargé d'études a son propre compte Postgres personnel) — voir la section « Modèle de déploiement de `mcp-postgres` : plan en 3 phases » ci-dessous, qui remplace cette hypothèse de centralisation immédiate.

Cette question est une sous-partie de la décision d'hébergement partagé déjà anticipée plus haut (« commencer en local... avant d'envisager un hébergement partagé ») — à trancher au moment de packager le déploiement pour plusieurs chargés d'études, pas avant. Aucun outil de supervision précis n'a été benchmarké à ce stade.

MODÈLE DE DÉPLOIEMENT DE MCP-POSTGRES : PLAN EN 3 PHASES (24/07/2026)

Suite à un échange avec Charlotte : centraliser `mcp-postgres` sur Merlin (comme envisagé ci-dessus) supposerait une identité de connexion partagée. Or aujourd'hui, chaque chargé d'études n'a que son propre compte Postgres personnel — centraliser obligerait à stocker tous ces mots de passe individuels dans une configuration gérée par l'admin sur Merlin, ce qui contredit l'intérêt même d'un identifiant personnel (contrôle du secret par la personne elle-même, pas par un tiers qui le détiendrait pour tout le monde à la fois).

Plan retenu, en trois phases :

1. **Phase 1 (actuelle)** : le serveur MCP Postgres tourne sur le poste individuel de chaque chargé d'études, avec son propre compte Postgres personnel — même montage que `mcp-qgis` (`mcpo` + connexion « Direct » personnelle, cf. section « pourquoi `mcpo` » ci-dessus). Deux implémentations testées en parallèle pour comparaison : `crystaldba/postgres-mcp` (déjà en place, `servers/mcp-postgres/`) et `bytebase/dbhub` en transport `stdio` (nouveau, `servers/mcp-dbhub/`) — voir `benchmark-techno.md` , section `mcp-postgres` , pour l'alerte sur l'état de maintenance de `crystaldba` qui motive cette comparaison. Choix pas encore tranché.
2. **Phase 2 (probable, pas encore engagée)** : créer un compte Postgres dédié à l'IA (compte de service partagé, en lecture restreinte) plutôt que de continuer sur des comptes personnels — permettrait de centraliser sur Merlin sans le problème de dépôt centralisé des mots de passe individuels décrit ci-dessus. La traçabilité par utilisateur serait alors portée par les logs d'Open WebUI (quel utilisateur, quel appel d'outil), pas par le compte Postgres.
3. **Phase 3 (plus tard)** : instances dédiées par utilisateur si un besoin de droits réellement différenciés par personne persiste malgré le compte de service de la phase 2 — piste `pgplex/pgconsole` (délégation « pour le compte de » un utilisateur, plafonnée à ses droits) ou équivalent, à réévaluer à ce moment-là (cf. `benchmark-techno.md` , section `mcp-postgres`).

A AJOUTER

Pour une vue où *chaque serveur MCP tourne physiquement* (poste agent vs serveur Docker centralisé), voir `schema-deploiement-prod.md` (même dossier).

à intégrer si ce n'est déjà fait : - pb des `mcpo` local (coût support) - fiabilité `mcp` à vérifier - mode projet remis à plus tard (`vscode` + `cline` ?) - cas d'usage 1 : bd pas encore documentée (pourra être mis en rag, étude metadonnées `geonetwork` à étudier)

LIMITES ET PERSPECTIVES

Cas d'usage 1 - Exploration base de données Pas encore de catalogue et chantiers d'intégration en cours : le prototype ciblera des tables en particulier : - base éco - MOS / Cosia - logement social

Cas d'usage 7 à 9 Le